ELEC 475 Computer Vision with Deep Learning

Lab 4 (Project) Report

Wallace Su, 20336688

Prof M. Greenspan

Date: December 2nd, 2025

1. Introduction

Traditional computer vision models are trained for narrow and task specific purposes, which depend heavily on large, labelled datasets. This means that they do not generalize well and require retraining for new tasks, which is both resource and time consuming. Traditional models typically do not relate images to language natively.

CLIP solves these problems by learning a shared multimodal embedding space where images and text are converted to vectors that are directly comparable to each other. Instead of predicting labels, CLIP uses contrastive learning to align matching image-caption pairs (pulling similar pairs closer and pushing non-similar pairs apart), which creates meaningful semantic representations that allow:

- Zero-shot learning: can perform tasks using natural language prompts (e.g., "a photo of a cat")
- Multimodal retrieval: can search images with text or text with images
- Transfer learning: can fine tune for specific domains (like the COCO dataset for this lab)
- Semantic understanding: connecting textual and visual meaning

CLIP has two encoders, one for images and one for text which map both inputs to the same 512-dimensional space:

- Image Encoder (trainable):
    - Uses a ResNet50 backbone ➔ 2048-dimensional features
    - Projection head ➔ 512-dimensional normalized embeddings
- Text Encoder (frozen):
    - Pretrained CLIP Transformer
    - Converts text tokens into 512-dimensional normalized embeddings

Both outputs lie on a unit hypersphere so that cosine similarity can be used to directly measure alignments of features.

CLIP is trained using InfoNCE (Information Noise Contrastive Estimation) which simultaneously:

- Pulls matching pairs closer together
- Pushes non-matching pairs further apart

For a batch of N pairs, each item uses others as negatives. The model computes image-to-text and text-to-image losses and averages them.

Some key training design choices that were made include:

- Symmetric contrastive loss: alignment in both directions
- L2 normalization: stable similarity comparisons
- Temperature parameter: controls sharpness of similarity distribution
- Frozen text encoder: preserves CLIP's language understanding while fine tuning the image encoder

In this lab, only the image encoder is fine-tuned to better match the fixed text embedding space on the COCO dataset used.

The data flow of the model is as follows:

1. Images ➔ ResNet50 ➔ Projection head ➔ 512-dimensional embeddings
2. Captions ➔ Frozen CLIP text encoder ➔ 512-dimensional embeddings
3. Compute similarity matrix
4. Apply InfoNCE contrastive loss
5. Backprop only through the image encoder

There are some key differences between the model created in this lab and the original CLIP. Some of the key differences include, a different image encoder (ResNet50), a frozen text encoder, fine-tuning image encoding only and the COCO 2014 dataset (rather than the 400M pairs used for the original CLIP).

The expected outcomes of this model were initially:

- Initial loss ~2.5-3.0 (random alignment)
- Trained model:
    - Positive image-text pairs show moderate-high similarity
    - Negative pairs show low similarity
    - Lower contrastive lost, higher retrieval recall

Such that the system should effectively align COCO images with their caption in a joint embedding space.

2. Methodology

2.1.1 Overview

The system fine tunes CLIP by training a ResNet50 image encoder to match the embedding space of a frozen CLIP text encoder. Both of these produce a 512-dimensional L2 (Euclidean) normalized embeddings which enables direct cosine similarity comparison.

2.1.2 Image Encoder

- Backbone: ResNet50 pretrained on ImageNet-1k
- Output: 2048-dimensional features ➔ projection head ➔ 512-dimensional embeddings
- Projection head:
    - Linear (2048 ➔ 512) ➔ GELU (Gaussian Error Linear Unit)
    - Linear (512 ➔ 512) ➔ L2 normalization
    - Optional: dropout, layer norm, batch norm
- Trainable parameters:
    - ResNet50: ~23.5M
    - Projection head: ~1.3M

2.1.3 Text Encoder

- Model: CLIP ViT-B/32 text encoder
- Status: frozen (no gradient updates)
- Output: 512-dimensional L2-normalized embeddings
- Purpose: Preserve CLIP's pretrained language understanding

2.1.4 Sharing Embedding Space

Both encoders map inputs to a unit sphere 512-dimensional space, enabling cosine similarity and contrastive learning. L2 normalization forces all embeddings to have unit length so that their magnitude becomes fixed, making their direction represent semantic information. This keeps the embedding distributions aligned and stable throughout training. Additionally, normalizing the outputs allows for the cosine similarity to become a computationally fast dot product. Contrastive learning also benefits on normalized unit hyperspheres as positive pairs need to be pushed directionally closer, rather than scaled up, preventing shortcut solutions where the model increases the vector magnitude rather than meaningful semantic direction.

2.2.1 Dataset

The dataset being used is Microsoft's COCO (Common Objects in Context) 2014, originally created to advance the state of computer vision by providing a robust benchmark for object detection, segmentation and image captioning. The dataset features images with multiple objects in real world contexts. The dataset includes:

- 82k training images
- 40k validation images
- Multiple captions per images

2.2.2 Preprocessing

To preprocess the image for the purposes of this lab, the following is done to images and text:

- Images
    - Resized to 224x224
    - CLIP normalization
    - Some other optional augments include: random crop, flip, color jitter
- Text
    - Tokenize with CLIP tokenizer (with a max length of 77)
    - Encode once with frozen text encoder
    - Cache 512-dimensional embeddings for efficiency

2.3.1 Contrastive Loss (InfoNCE)

Training minimizes symmetric InfoNCE loss that:

- Pulls matched image-text pairs together
- Pushes mismatched pairs apart

- Uses temperature τ = 0.07, which is a scaling factor that controls how sharp or soft the distribution is. A value of 0.07 is relatively small which makes the distribution sharper, making the model aggressive in distinguishing positives from negatives, forcing strong contrast.

2.3.2 Hyperparameters

The hyperparameters of the model are as follows:

- Batch size: 32
- Learning rate: 1^{-4}
- Optimizer: AdamW
- Weight deacy: 0.01
- Learning Rate Schedule
    - Baseline: CosineAnnealingLR
        - T_max: 10 epochs
        - eta_min: 1^{-6}
    - With warmup: Linear warmup for 2 epochs, then cosine annealing
- Temperature (τ): 0.07
- Number of Epochs: 10
- Gradient Clipping: max_norm = 1.0 (for stability)
- Number of workers: 4 (for data loading)

2.3.3 Training Procedure

1. Forward Pass
    a. Images ➔ ResNet50 ➔ Projection Head ➔ Image Embeddings (512-dim)
    b. Text ➔ Cached Text Embeddings (512-dim)
2. Loss Computation
    a. Compute similarity matrix: [batch_size x batch_size]
    b. Apply InfoNCE loss (symmetric in both directions)
3. Backward Pass
    a. Compute gradients for image encoder only
    b. Apply gradient clipping (max_norm = 1.0)
    c. Update image encoder parameters
4. Validation
    a. Evaluate on validation set after each epoch
    b. No data augmentation during validation
    c. Save best model based on validation loss
5. Model Selection
    a. Best model is selected based on lowest validation loss
    b. A checkpoint is saved after each epoch is validation loss improves

2.3.4 Optional Modifications (Ablation Study)

Some optional modifications and their perceived benefits for the ablation study included:

- Augmentation – stronger image transforms
- Layer_norm – stabalizes projection head
- Batch_norm – alternative normalization
- Dropout – regularization
- Warmup – learning learning rate warmup before cosine decay
- All – applying every modification together in one training session

In the code, each configuration is trained separately in order to have a fair comparison with the baseline trained model (with no modifications).

2.4.1 Evaluation Metrics

The following metrics are measured on the COCO validation using retrieval

- Image ➔ Text and Text ➔ Image
    - Recall @1, @5, @10
    - Mean rank

2.4.2 Evaluation Process

1. Compute embeddings for all validation images and captions
2. Build a cosine similarity matrix (N x N)
3. Rank captions/images
4. Compute Recall@K + mean rank

2.4.3 Additional Qualitative Checks

Along with Recall@K and mean rank, the following qualitative checks are performed to ensure validity of trained models

- Text-to-image retrieval visualizations
- Zero-shot classification using text prompts

2.1.1 Implementation Details

The hardware used for the training is a Nvidia RTX 3080 which took roughly ~60-70 minutes for 10 epochs, with a ~10% increase in training time between the different augmentations and modifications. The best checkpoints were saved using validation loss while training to ensure that later epochs produced more successful results, rather than regressing.

3. Results

3.1.1 Baseline Configuration

The following are the training results of the baseline configuration

- Train loss: 0.657 ➔ 0.100 over 10 epochs
- Validation loss: 0.949 ➔ 0.856 (at epoch 5), which then rises slightly to 0.877 at epoch 10
- Time: ~73 minutes total

The rapid drop in training loss shows that the model can fit the COCO training data well. The validation loss plateau and slight increase after epoch 4-5 indicate slight overfitting where the image encoder keeps fitting the trained pairs but does not improve on unseen data. The widening gap (0.100 train vs 0.877 validation) confirms that the baseline has high capacity but limited regularization.

3.1.2 Data Augmentation Configuration

The following are the results of training with the data augmentation configuration:

- Train loss: 0.681 ➔ 0.224
- Validation loss: 0.970 ➔ best of 0.806 (at epoch 9)
- Time: ~74 minutes total

The training loss is higher than the baseline at each epoch, which is expected behaviour as data augmentation makes the task harder, preventing overfitting. The validation loss consistently decreases and reaches 0.806 at its best, a 5.8% improvement over the baseline best (0.856).

The smaller train-validation gap (0.224 vs 0.808) indicates that the trained model with augmentation has better generalization and no longer purely memorizes image-caption pairs, forcing it to learn more robust representations. Augmentation adds minimal computational overhead, making this a high impact, yet low-cost modification.

3.1.3 All Modifications Combined

This is the results of all modifications combined, which include augmentation, layer normalization, dropout and warmup:

- Train loss: 1.478 ➔ 0.240
- Validation loss: 1.116 ➔ best of 0.812 (at epoch 9)
- Time: ~67 minutes total

The very high initial loss reflects the combined modifications and the more cautious initial optimization. Despite this, training loss drops significantly to 0.240 and the validation loss improves steadily to 0.812. The best validation loss (0.812) is relatively close to augmentation-only (0.806) but not better, which indicates diminishing returns when stacking modifications. The reduced training time indicates that normalization and scheduling can improve training efficiency.

3.1.5 Summary of Configurations

Config	Final Train Loss	Best Validation Loss	Relative Improvement to Baseline	Total Time (Minutes)
Baseline	0.100	0.856	-	73.0
Augmentation	0.224	0.806	5.8% (better)	73.8
Layer Norm	0.103	0.854	Roughly equal	62.1
Batch Norm	0.107	0.860	Slightly worse	62.2
Dropout	0.110	0.860	Slightly worse	63.0
Warmup	0.097	0.865	Worse	63.2

All	0.240	0.812	5.1% (better)	66.5

The key takeaways of this chart are that data augmentation was the most effective single change, with best validation loss and reduced overfitting. The layer norm showed small improvements and faster training, while batch norm, dropout and warmup showed slightly negative impacts, indicating that the model may already be sufficiently regularized. With all of them combined, it is shown that stacking regularization helps but not much beyond just augmentation alone.

3.2.1 Image-to-Text Retrieval

All the following retrieval metrics reported are for the baseline model without augmentation or modifications. Given an image, rank all captions:

- Recall@1: 16.64%
- Recall@5: 39.82%
- Recall@10: 53.52%
- Mean rank: 34.45

This means that over half of the images have the correct caption in the top 10 results, showing that the shared embedding space is reasonably aligned. A Recall@1 of 16.64% indicates that that exact top 1 matching is still challenging, especially on a noisy, multi-caption dataset like COCO. A mean rank of ~34 means that on average the correct caption is within the top few dozen results.

3.2.2 Text-to-Image Retrieval

Given a caption, rank all images:

- Recall@1: 21.72%
- Recall@5: 48.42%
- Recall@10: 61.92%
- Mean rank: 30.39

From these Recall@K numbers, it is evident that text-to-image beats image-to-text, as over 60% of captions retrieve the correct image in the top 10, a strong metric considering the diversity of the COCO dataset. A lower rank means that the model is slightly better at mapping text ➔ images than images ➔ text.

3.2.3 Retrieval Summary

The model achieves usable retrieval quality, especially when users see top 5 to top 10 results. The model shows a weakness in determining the top candidate but generally has the right answer within the top few candidates.

3.3.1 Validation Loss and Generalization

From the ablation table:

- Augmentation: 0.806 (best overall)
- All modifications: 0.812

Data augmentation clearly states itself as the most impactful change; it improves generalization without major overhead and directly reduces overfitting seen in the baseline. Layer norm helps marginally, indicating that stabilizing the projection head can help but its effect is modest compared to augmentation.

4. Discussion

There are a few key takeaways that can be interpreted and analyzed. Looking at the results and qualitative results based on different modifications, augments and the baseline can help determine weak spots in the model's behaviour and performance. It is known that computational power is one of the main limiting factors of machine learning and computer vision, so finding ways to analyze the system and optimize computational expenses is a crucial part of learning.

After looking at results, it's easy to see that overfitting is a core problem of the system. This can be seen where the training loss drops sharply to 0.100 while validation loss bottoms out around 0.0856 and the rises. The wide train-validation gap after ~epoch 4 shows that the image encoder is memorizing training pairs rather than learning transferable features. This behaviour is expected in a fine-tuning setup with a high-capacity ResNet50 backbone, a small dataset (relative to CLIP's original scale) and a frozen, high-quality text encoder that doesn't adapt in training. This means that the overall challenge is generalization and not optimization stability.

As can be seen by the ablation study, data augmentation is the single most impactful modification. This can be seen by the validation loss numbers, improving from 0.856 to 0.806, showing a 5.8% loss reduction. Training loss remains higher (0.224 vs 0.100) which is a desirable as it indicates less memorization. This is because augmentation increases input diversity and prevents the model from seeing identical images on each epoch. It forces the encoder to learn more invariant, semantic features which directly attacks the root cause of overfitting. The computational costs are nearly identical, so this makes augmentation very beneficial yet low costing.

Moving on, modifications like layer norm, batch norm, dropout and warmup only provide marginal or very minimal improvements. Layer norm improves validation but not dramatically. Batch norm, dropout, and warmup are on par or slightly worse than the baseline in validation performance. These modifications can make optimization a little smoother and faster, but they don't do anything to address overfitting. With all models combined, the validation loss is 0.812 which is close to the validation loss of the augmentation-only model of 0.806. This shows a diminishing return and proves that augmentation is the dominant factor in solving overfitting.

Retrieval metrics show consistent asymmetry as T2I outperforms I2T in all Recall@K and mean rank metrics. The interpretation of this is that the text encoder is frozen CLIP and pretrained on a massive dataset – meaning that text embeddings are well structured and discriminative. In contrast the image encoder is only fine-tuned on COCO and its embeddings must adapt to the fixed text space. This suggests the learned embedding space has good global alignment (Recall@10), but fine-grained ranking (Recall@1) and image-side representation quality are the main bottlenecks.

The embedding space and the loss curves reveal a few key things about the model. Firstly, they show that the global structure is good – a high Recall@10 and a decent mean rank imply positive pairs are somewhat near each other in the embedding space. In contrast, fine-grained separation is

weak – a low Recall@1 shows the model often confuses similar candidates. Additionally, InfoNCE loss decreases steadily but eventually plateaus, indicating that the model has learned to distinguish positives from obvious negatives, but struggles to make more subtle distinctions.

5. Conclusion

After conducting this lab and various experiments, such as the ablation study, the central empirical findings were:

- Data augmentation is the single most effective modification, improving best validation loss by roughly ~5.8% over the baseline.
- The baseline model overfits strongly with a large train-validation loss gap
- T2I retrieval outperforms I2T, reflecting the asymmetry cause by a frozen, high-quality text encoder combined with a fine-tuned image encoder.
- Combining multiple regularization techniques yields diminishing returns, with augmentation alone matching all modifications applied together.

After conducting the experiments in this lab, what was learned?

- Hands on experience with InfoNCE
    - How +/- pairs shape the embedding space
    - Role of temperature scaling
    - How contrastive training differs from typical label prediction
- Fine-tuning large pretrained models
    - Freezing the text encoder while tuning the image encoder
        - Creates performances asymmetry
    - Overfitting is a primary failure mode when fine tuning on relatively small datasets like COCO
    - Training loss alone is misleading, should always be compared to validation loss and retrieval metrics
- Regularization hierarchy
    - Data augmentations are more impactful than architectural tweaks (dropout, norm, warmup) for this task

This lab also demonstrated the value of ablation studies – intuition can often be wrong or incomplete in practice. Testing each modification in isolation is crucial for understanding what matters. Metrics and visualizations like Recall@K and loss curves are also essential in analyzing results as they put abstract numbers and figures into digestible formats, which are essential for improving and optimizing the model.

Some things that worked well included:

- Data augmentation was the standout success
- The ablation framework worked well
- Saving full training histories enabled more quantitative metrics

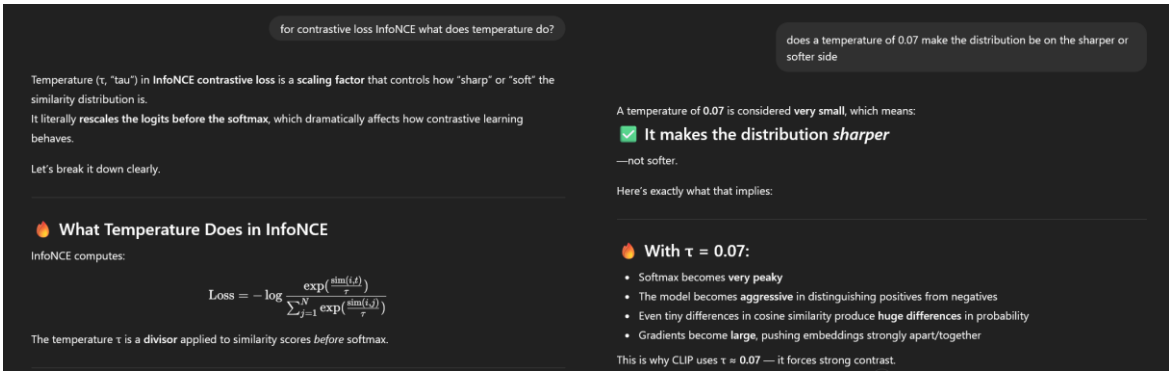
In contrast, some key areas of improvement for next can include:

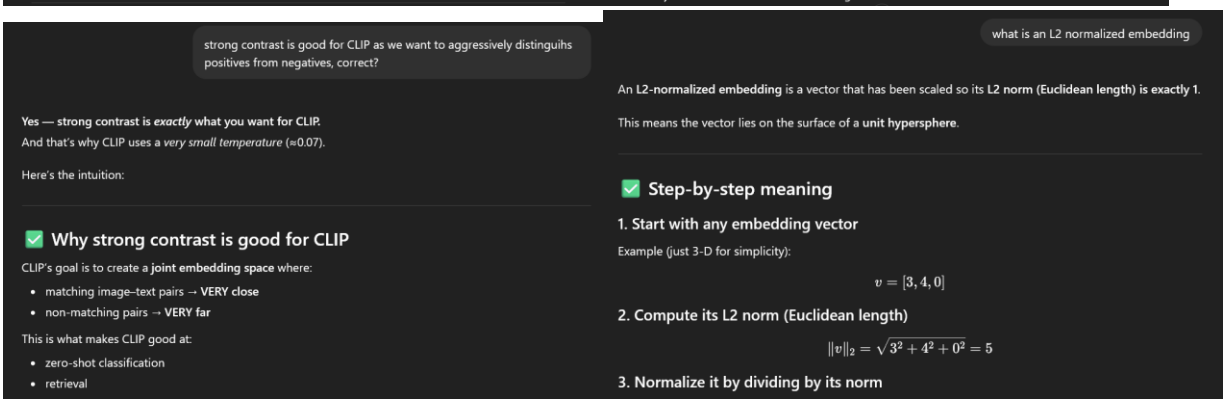
- Extended and better-tuned training
    - Train for more epochs (if time and computational resources exist) with proper early stopping to check if Recall@1 and validation loss continue to improve
    - Perform hyperparameter tuning per configuration instead of one set for all
- Architecture and representation
    - Explore fine tuning both encoders to reduce asymmetry between I2T and T2I
- Stronger regularization and training
    - Try stronger augmentation
    - Test larger batch sizes (which means more negatives per batch) to enhance contrastive learning
    - Revisit modifications like dropout, warmup and normalization using these

All in all, this lab illustrates some general lessons for multimodal and contrastive learning, the main one being: solve the right problem first. In this lab, the right problem was clearly overfitting which could have been solved with additional and more powerful data augmentation.

6. Use of LLMs

I used LLMs such as ChatGPT to help clarify some of the topics covered in this lab as well as ask it to clarify questions whenever I was confused or stuck on a concept. For example, I asked it things like “why does data augmentation help reduce overfitting while other modifications like batch norms not” or “why do we have to put the embeddings into a unit hypersphere”. After asking these questions I also like to ask it to clarify questions while tagging “correct?” at the end to ensure my understanding is not mistaken. I cannot attach a link to the specific chats as I primarily used Cursor, but I can provide various screenshots of questions I asked the LLMs below:





7. Appendix

Attached below will be some useful figures, such as loss curves and examples of the model, such as I2T and T2I.

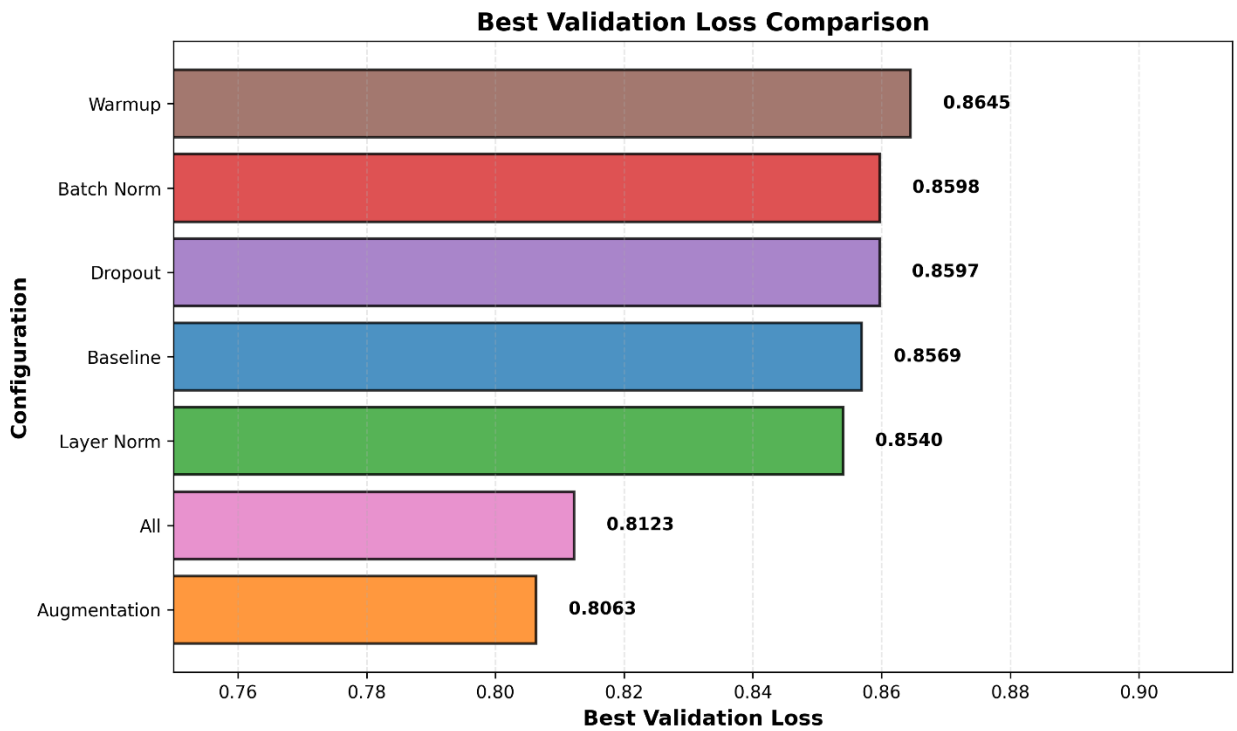


Figure 1: Bar graph of validation loss of different modifications

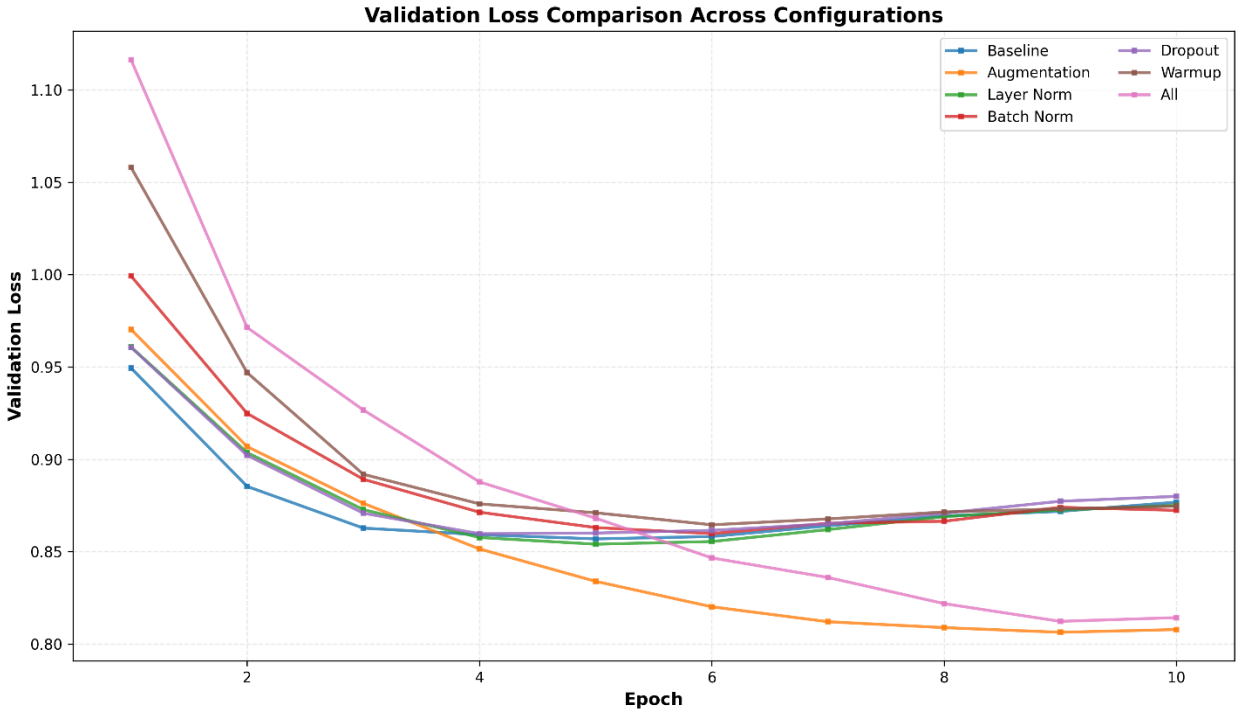


Figure 2: Validation loss of different modifications across epochs

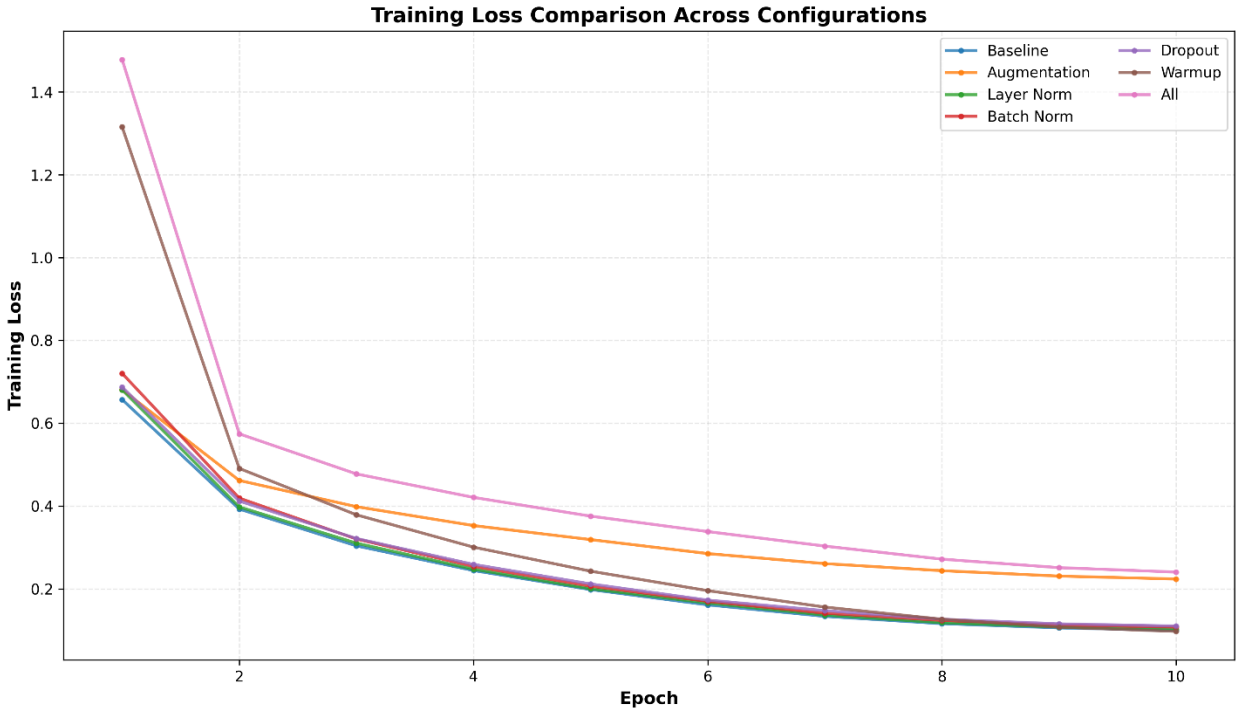


Figure 3: Training loss of different modifications across epochs

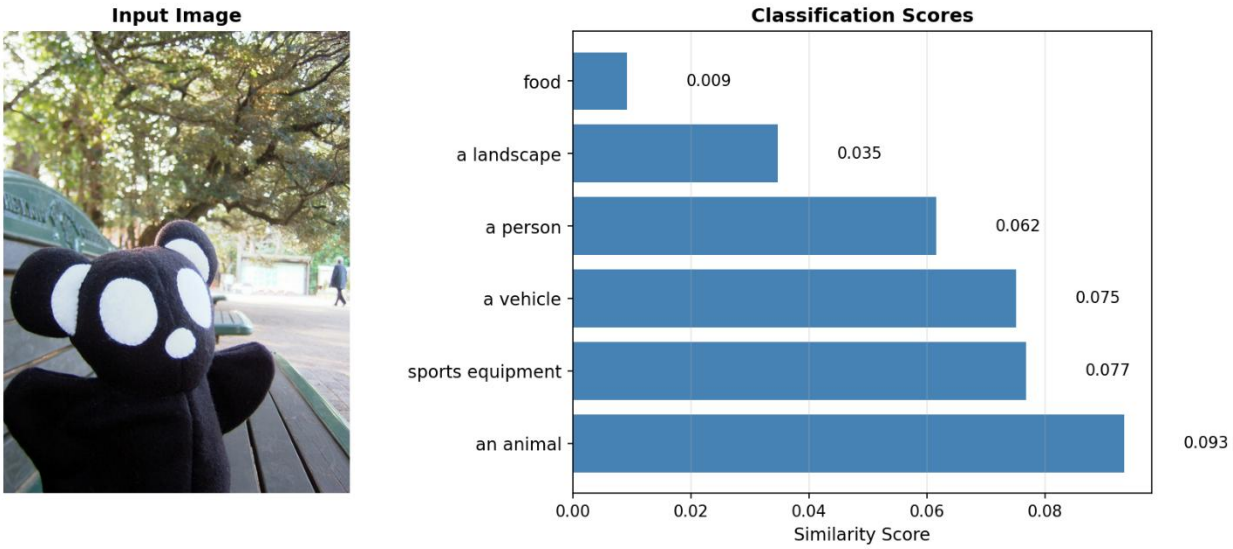


Figure 4: I2T example 1

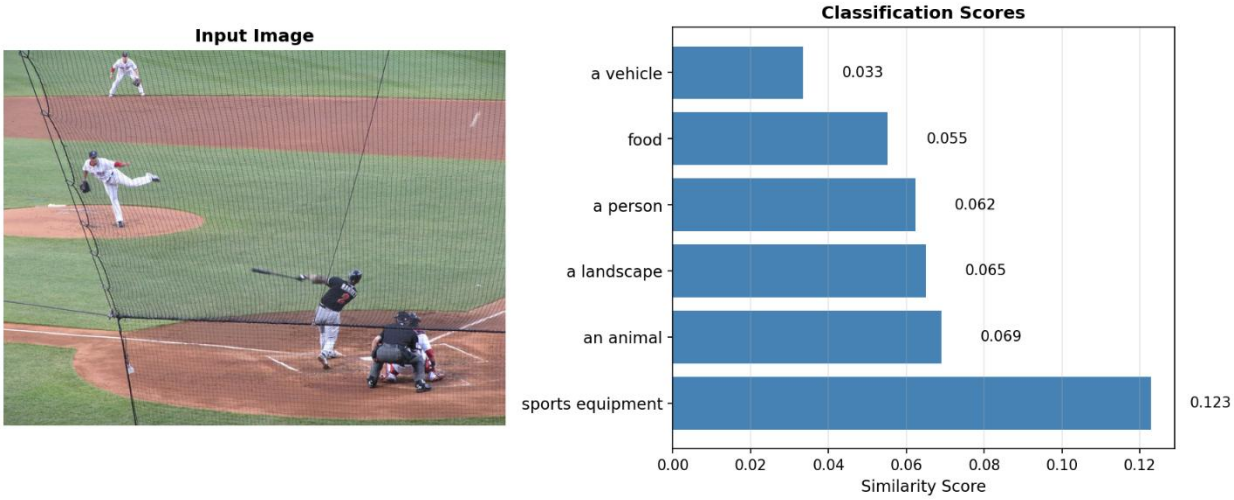


Figure 5: I2T example 2

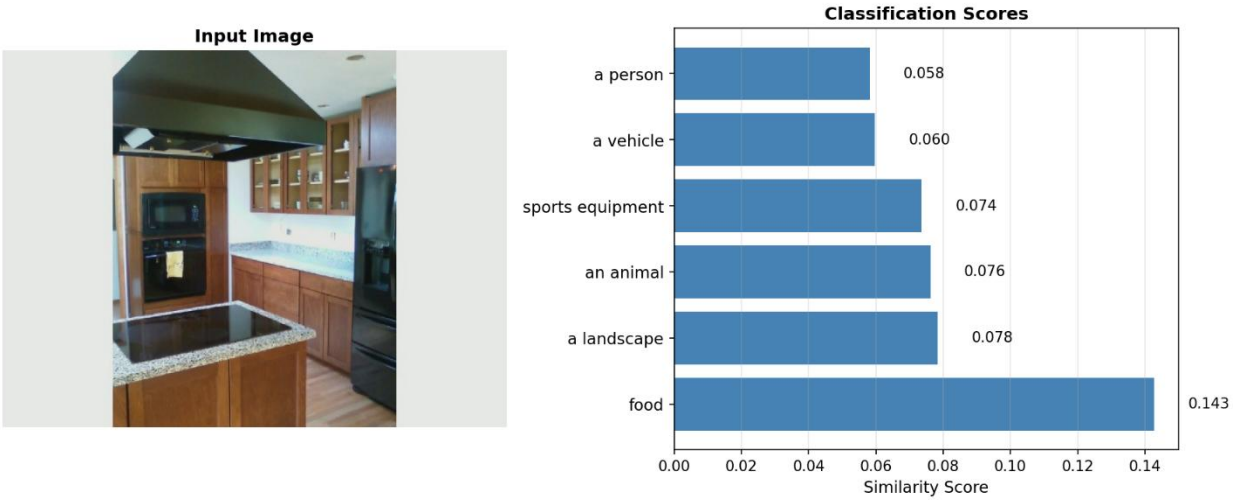


Figure 6: I2T example 3

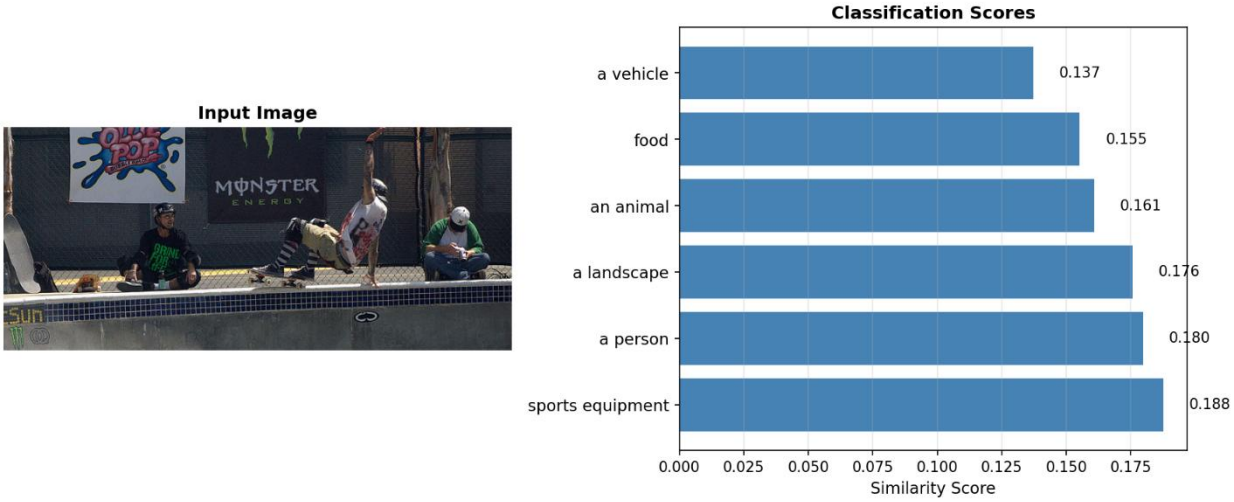


Figure 7: I2T example 4

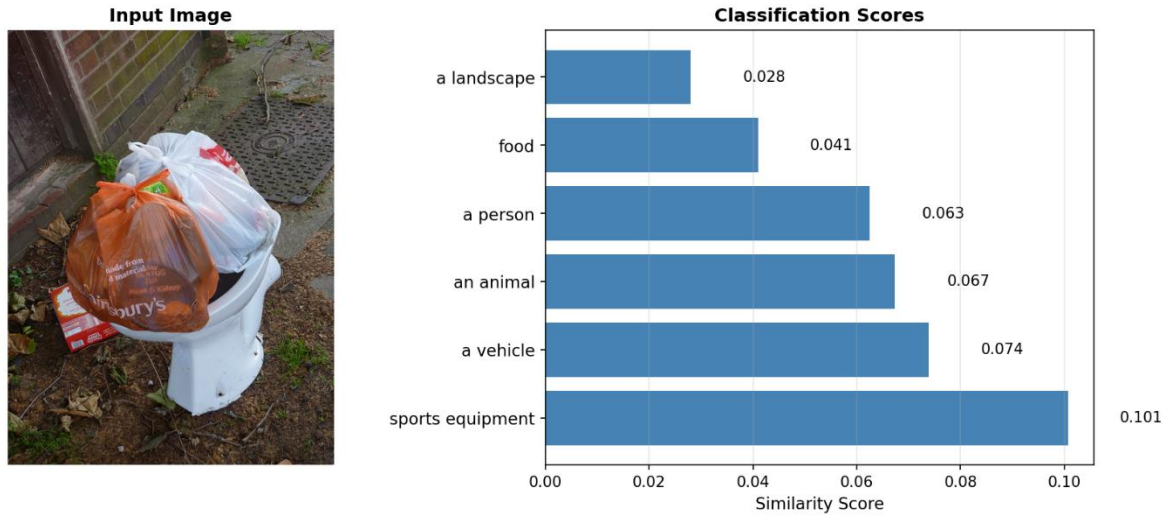


Figure 8: I2T example 5

Figure 9: T2I example 1

Figure 10: T2I example 2

Figure 11: T2I example 3